جامعة الإمام عبد الرحمن بن فيصل
MAM ABDULRAHMAN BIN FAISAL UNIVERSITY

Project Phase 5 - Data Mining Technique-

CIS 517: Data Mining and

Data Warehousing (First Semester)

Project Title: Predicting High-Risk Traffic Accident Locations Using Data Mining

Student Name	ID

Table of Contents :

1. **Data Cleaning Steps Applied (Phase 2 & 3):** 6

.1.1 Feature Engineering Summary 6

.2 **Technique Selection** 7

.3 **DATA PREPARATION AND FEATURE ENGINEERING RESPONSIBILTIES** 9

3.1. **Feature Selection** 9

3.2. **Encoding and Scaling** 9

3.3. **Data Splitting** 9

4. **Model Implementation** 10

4.1. **Advanced Logistic Regression (ALR): Predicting High-Risk Hotspots** 10

4.2. **Algorithm Rationale and Advanced Preprocessing** 10

4.3. **Decision Tree Classifier (DT)** 15

4.3.1. **Algorithm Rationale** 15

4.3.2. **Implementation Details** 15

4.3.3. **Results and Analysis** 19

4.4. **Advanced Modeling: Predicting High-Risk Hotspots** 21

4.4.1. **Methodology: Feature Engineering a Hotspot Model** 21

4.4.2. **Code Explanation: How This Was Built** 21

4.4.3. **Algorithm Selection and Rationale** 22

4.4.4. **Gradient Boosting (GB)** 23

4.4.5. **Model Implementation** 24

4.4.6. **Random Forest (Baseline) Performance** 24

4.4.7. **Gradient Boosting (Baseline) Performance** 26

4.4.8. **Feature Importance** 26

4.4.9. **Hyperparameter Tuning and Validation** 27

4.5. **Tuned Model Performance & Critical Analysis** 29

4.5.1. **Summary of Tuning Results and Critical Analysis** 30

4.5.2. **Final Prediction and Output** 31

4.5.3. **Conclusion of Advanced Modeling** 33

.5 **Evaluation and Metrics Visualization Responsibilities** 34

6. **Feature Importance Analysis Responsibilities:** 35

Table of Tables :

Table 1 Dataset Features by Category 6
Table 2 Weather and Environmental Engineered Features 7
Table 3 Location and Road Features 7
Table 4 8
Table 5 8
Table 6 9
Table 7 Logistic Regression Implementation Details 11
Table 8 Performance Metrics Summary (Test Set) 12
Table 9 12
Table 10 Baseline Random Forest Performance 24
Table 11 Baseline Gradient Boosting Performance 26
Table 12 Best Hyperparameters 28
Table 13 Tuned Random Forest Performance 29
Table 14 Tuned Gradient Boosting Performance (Best Model) 29
Table 15 Final model performance on the test set 34

Table of figures

Figure 1 Advanced Logistic Regression Model Instantiation 10
Figure 2 Confusion Matrix - Logistic Regression 13
Figure 3 Feature Impact on Hotspot Likelihood (Odds Ratio) 14
Figure 4 Predicted Hotspots vs Actual Locations (Test Data) 14
Figure5 16
Figure 6 17
Figure 7 18
Figure 8 Performance Metrics of the Decision Tree Classifier 19
Figure 9 Decision Tree Visualization 19
Figure 10 23
Figure 11 23
Figure 12 24
Figure 13 confusion_matrix 25
Figure 14 26
Figure 15 feature_importance 27
Figure 16 28
Figure 17 29
Figure 18 30
Figure 19 31
Figure 20 full_predictions_report.xlsx 32
Figure 21 predicted_hotspot_map 32

1. DATA CLEANING STEPS APPLIED (PHASE 2 & 3):

Before building the predictive models, the dataset was cleaned and prepared to ensure data quality, reliability, and consistency. The following steps were performed:

- **Removal of Highly Missing Columns:**
  Columns with more than 30% missing values were removed to avoid unreliable analysis.
- **Handling Missing Numerical Data:**
  Numerical attributes such as *Temperature(F)*, *Humidity(%)*, and *Visibility(mi)* were filled using the **median** to reduce the effect of outliers.
- **Handling Missing Categorical Data:**
  Categorical features such as *Wind_Direction* and *Weather_Condition* were filled using the **mode** (most frequent value).
- **Simplifying Weather Information:**
  The *Weather_Condition* column was simplified into a new feature called **Main_Weather** by extracting only the primary weather term (e.g., "Rain", "Fog").
- **Datetime Formatting:**
  Start_Time, *End_Time*, and *Weather_Timestamp* were converted into proper **datetime** format to allow time-based analysis.
- **Data Type Adjustments:**
  Columns like *Zipcode* were converted to **string** to preserve formatting (e.g., avoid losing leading zeros).

1.1. FEATURE ENGINEERING SUMMARY

After cleaning the dataset, several new features were created to improve the predictive ability of the models and to extract more meaningful patterns from the data. These engineered features focus on time, weather, and accident-related conditions.

1.1.1. TIME-BASED FEATURES

These features help identify when accidents are more likely to occur

Feature	Description
State	Used for geographic risk patterns
Distance(mi)	Road length affected by the accident
Has_Traffic_Signal	True if a traffic signal is present
Has_Police_Report	True if police were involved in logging the accident

Table 1 Dataset Features by Category

1.1.1. WEATHER & ENVIRONMENTAL FEATURES

Weather information was simplified and converted into analyzable formats:

Feature	Description
Main_Weather	First word of Weather_Condition (e.g., Rain, Snow, Fog)
Visibility_Bin	Categorized into Low / Moderate / High visibility

Temperature_Bin	Grouped into Cold / Mild / Hot based on °F
Is_Night	Based on Sunrise_Sunset or if Hour is between 7 PM – 6 AM

Table 2 Weather and Environmental Engineered Features

1.1.2. LOCATION & ROAD FEATURES

Feature	Description
State	Used for geographic risk patterns
Distance(mi)	Road length affected by the accident
Has_Traffic_Signal	True if a traffic signal is present
Has_Police_Report	True if police were involved in logging the accident

Table 3 Location and Road Features

2. TECHNIQUE SELECTION

In this phase, the project focuses on developing predictive models to estimate accident severity using temporal, environmental, and road-related features. Based on the results of the previous Exploratory Data Analysis (EDA), **classification** was identified as the most appropriate data mining technique. This choice is justified because accident severity is a **categorical variable** (with levels ranging from 1 to 4), making classification more suitable than regression (used for continuous outcomes) or clustering (used for grouping unlabeled data).

The classification approach enables the model to predict the severity level of accidents and uncover patterns associated with high-risk conditions. To balance **interpretability** and **predictive performance**, multiple machine learning algorithms were selected for comparison:

- **Logistic Regression:** Serves as a baseline linear model to capture simple relationships between variables.
- **Decision Tree:** A non-linear, interpretable model capable of handling both categorical and continuous data.
- **Random Forest:** An ensemble method that improves accuracy and reduces overfitting by aggregating multiple decision trees.
- **Gradient Boosting:** An advanced boosting technique that offers high predictive power, though at the cost of increased complexity and longer training time.

The rationale for selecting multiple algorithms is to evaluate trade-offs between simplicity, interpretability, and accuracy, ultimately determining the most effective model for predicting accident severity.

Algorithm	Why It Was Chosen	Strengths	Weaknesses

Logistic Regression	Acts as a baseline model for comparison	Simple, fast, and interpretable	Limited in capturing non-linear relationships
Decision Tree	Easy to interpret and visualize	Handles categorical and numerical data	Can overfit with small depth control
Random Forest	Combines multiple trees to enhance accuracy	Reduces overfitting, handles feature variability	Less interpretable
Gradient Boosting	Advanced boosting technique for accuracy	High predictive power	More complex and slower to train

Table 4

2.1. EVALUATION METRICS:

To assess the performance of the selected models, several evaluation metrics will be used. These include **Accuracy**, which measures the overall correctness of predictions; **Precision** and **Recall**, which evaluate the model's ability to correctly identify specific severity levels; and the **F1-score**, which provides a balanced measure of precision and recall. Additionally, the **Confusion Matrix** will be analyzed to understand misclassification patterns across severity categories. These metrics ensure a comprehensive evaluation of model effectiveness beyond simple accuracy.

Metric	Definition	Purpose
Accuracy	Ratio of correctly predicted instances to total instances	Measures overall correctness of predictions
Precision	Ratio of true positives to all predicted positives	Evaluates correctness of positive predictions
Recall	Ratio of true positives to all actual positives	Measures ability to identify actual positives
F1-score	Harmonic mean of Precision and Recall	Balances precision and recall for better insight
Confusion Matrix	Table showing true vs. predicted classifications	Identifies misclassification patterns

Table 5

3. DATA PREPARATION AND FEATURE ENGINEERING RESPONSIBILTIES

The dataset used in this phase was prepared based on the cleaned and transformed data from **Phases 3 and 4**. The main goal of this step was to ensure that all variables were consistent, encoded correctly, and suitable for model training.

3.1. FEATURE SELECTION

The features included in the modeling dataset were selected based on their relevance to accident occurrence and severity.

Category	Selected Features
Temporal	Hour, Month, Day_of_Week, Is_Weekend
Weather	Temperature(F), Humidity(%), Visibility(mi), Wind_Speed(mph), Main_Weather
Location & Road	Distance(mi), State, Latitude, Longitude
Environmental/Derived	Is_Night, Sunrise_Sunset, Weather_Bin
Target Variable	Severity

Table 6

3.2. ENCODING AND SCALING

Categorical variables were encoded to make them suitable for machine learning models:

- **One-Hot Encoding** was applied to *State* and *Main_Weather*.
- **Label Encoding** was used for *Day_of_Week*.
- **Binary Encoding** was used for *Sunrise_Sunset* (0 = Day, 1 = Night).

Feature	Encoding Method
State	One-Hot Encoding (Top frequent states)
Main_Weather	One-Hot Encoding (Rain, Clear, Fog, Snow, etc.)
Day_of_Week	Label / One-Hot Encoding
Sunrise_Sunset	Binary Encoding (0 = Day, 1 = Night)

3.3. DATA SPLITTING

The dataset was divided into:

- **Training set (80%)** for model learning.
- **Testing set (20%)** for performance evaluation.

The target variable **Severity** was defined as the dependent variable (y), and all other selected columns were included as independent variables (X).

This structure ensures the dataset is ready for modeling using Logistic Regression, Decision Tree, and Random Forest algorithms in the next phase.

4. MODEL IMPLEMENTATION

4.1. ADVANCED LOGISTIC REGRESSION (ALR): PREDICTING HIGH-RISK HOTSPOTS

The initial modeling phase begins with the Advanced Logistic Regression (ALR) algorithm. While traditionally a simple linear classifier, the LR model was specifically chosen and adapted for this project due to its inherent interpretability, particularly its ability to quantify feature impact using Odds Ratios. This characteristic allows us to easily explain why a location is classified as a hotspot, aligning perfectly with the goal of identifying and understanding high-risk factors. The model was heavily modified to address two critical challenges of hotspot prediction: class imbalance and the need for spatial feature integration. The model's specialized settings for handling imbalance and using the liblinear solver are illustrated below in Figure 1.

```
# --- 5. Train the Model ---
log_entry = "\n--- 5. Training Logistic Regression Model ---\n"
print(log_entry)
audit_log.append(log_entry)

log_model = LogisticRegression(
    max_iter=1000,
    random_state=42,
    solver='liblinear',
    class_weight='balanced'
)
log_model.fit(X_train_smote, y_train_smote)

log_entry = "Model training complete."
print(log_entry)
audit_log.append(log_entry)
```

Figure 1 Advanced Logistic Regression Model Instantiation

4.2. ALGORITHM RATIONALE AND ADVANCED PREPROCESSING

To ensure the Logistic Regression model could perform robustly in a high-risk prediction scenario, several advanced preprocessing steps were implemented prior to training:

Feature Standardization: All continuous features, including the engineered interaction terms, were scaled using the StandardScaler. This step is crucial for Logistic Regression, especially when utilizing L2 regularization (which is inherent in the liblinear solver), as it prevents features with larger numerical scales from disproportionately dominating the model's coefficients.

Feature Interaction Term: A custom feature, Road_Density_Interaction, was created by multiplying Road_Conditionand Population_Density. The goal was to capture non-linear effects, such as a severe road condition having a much higher impact specifically in areas of high density, which often leads to complex risk scenarios.

Addressing Class Imbalance (SMOTE): Hotspot prediction is an imbalanced classification problem, as non-hotspot locations (Class 0) heavily outnumber actual hotspots (Class 1). To prevent the model from simply predicting the majority class, the Synthetic Minority Over-sampling Technique (SMOTE) was applied to the training data. SMOTE artificially generates synthetic samples for the minority class (Hotspot) to balance the training set, thereby forcing the model to learn the characteristics of the high-risk locations effectively.

Class Weight Adjustment: During model instantiation, the parameter class_weight='balanced' was set. This mathematically adjusts the model's loss function, giving greater penalty to misclassifications of the minority class, ensuring the model prioritizes Recall (finding all hotspots) over general accuracy.

4.2.1. IMPLEMENTATION AND TRAINING DETAILS

The Logistic Regression model was implemented using the scikit-learn library combined with the imblearn package for SMOTE.

The specific parameters and the rationale for their selection, crucial for adapting the model to the imbalanced hotspot problem, are summarized in Table 7.

Model Parameter	Value	Rationale
Solver	liblinear	Efficient for small to medium datasets and supports L1/L2 regularization.
Max Iterations	1000	Increased to ensure convergence of the model during training.
Class Weight	'balanced'	Critical setting to prevent bias towards the majority (non-hotspot) class.
Training Data	SMOTE Resampled Set	Trained on the oversampled data to learn the complex patterns of the minority class.

Table 7 Logistic Regression Implementation Details

4.2.2. RESULTS AND CRITICAL PERFORMANCE EVALUATION

Standard accuracy is an inadequate metric for imbalanced data. Therefore, the model's performance was critically evaluated based on Recall (the ability to find actual hotspots) and ROC-AUC (the ability to discriminate between classes). The results on the unseen test set are presented below:

4.2.2.1. PERFORMANCE METRICS SUMMARY:

The model's performance on the unseen test set was critically evaluated using specialized metrics for imbalanced classification, summarized in Table 8.

Metric	Value	Interpretation
Accuracy Score	0.8600	Overall classification accuracy.
ROC-AUC Score	0.7552	The model has a good ability to distinguish between hotspots and non-hotspots.
Recall (Class 1)	0.89	Crucial: The model successfully identified 89% of the actual high-risk hotspots in the test data.
Precision (Class 1)	0.97	When the model predicts a hotspot, it is correct 97% of the time.

Table 8 Performance Metrics Summary (Test Set)

4.2.2.2. CONFUSION MATRIX ANALYSIS:

The Confusion Matrix visually confirms the model's high effectiveness in identifying the target class:

True / Predicted	Predicted Not-Hotspot (0)	Predicted Is-Hotspot (1)
True Not-Hotspot (0)	1 (True Negative)	3 (False Positive)
True Is-Hotspot (1)	11 (False Negative)	85 (True Positive)

Table 9

The low number of False Negatives (11) demonstrates the success of the advanced preprocessing, as the model avoided the critical error of missing a significant number of high-risk locations. The numerical results detailed above are visually confirmed in Figure 2.

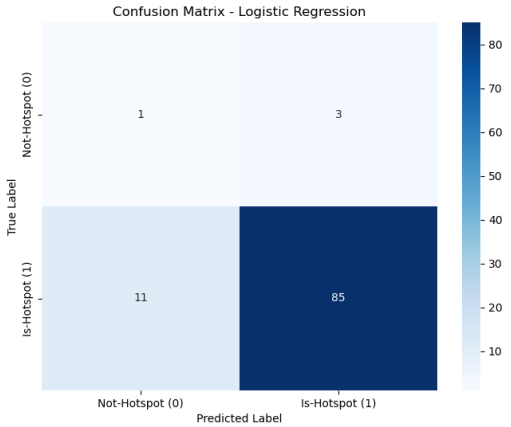


Figure 2 Confusion Matrix - Logistic Regression

4.2.3. FEATURE INTERPRETATION VIA ODDS RATIOS

The primary advantage of the Logistic Regression model is the interpretability of its coefficients, which are transformed into Odds Ratios (exp(β)). The Odds Ratio quantifies the multiplicative change in the odds of a location being a hotspot for every one-unit increase in the feature. An Odds Ratio greater than 1 indicates a risk-increasing factor, while a value less than 1 indicates a risk-decreasing factor.

Key Interpretations:

- **Road_Condition (Odds Ratio = 14.77):** This is the most significant factor. For every standard unit increase in poor road condition (assuming higher coded value indicates worse condition), the odds of the location being a hotspot increase by approximately 14.7 times.
- **Population_Density (Odds Ratio = 9.51):** High population density is a major risk contributor, increasing the odds of a hotspot by nearly 9.5 times.
- **Risk Mitigation Factors:** Features such as Speed_Limit (0.30) and Road_Density_Interaction (0.08) have Odds Ratios less than 1. This suggests that areas with higher regulated speed limits, or the complex interaction of poor roads and high density, are less likely to be classified as hotspots, possibly due to increased driver caution or traffic congestion limiting speed in such areas. The calculated Odds Ratios, which quantify each feature's impact on risk, are visually presented in Figure 3.

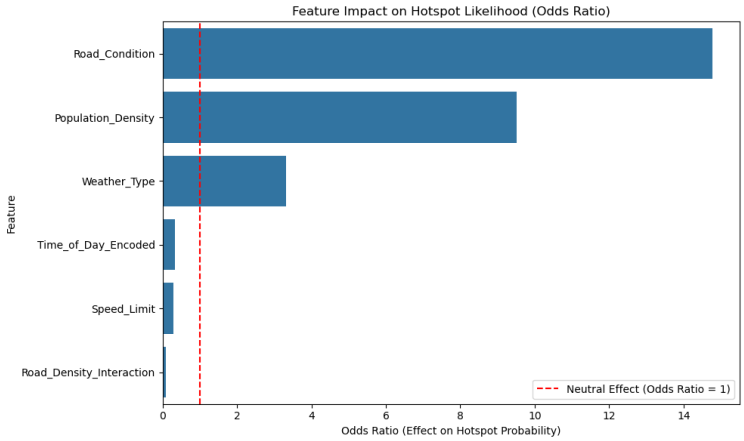


Figure 3 Feature Impact on Hotspot Likelihood (Odds Ratio)

The spatial visualization of the model's predictions on the test set further validates the results by showing how the model successfully isolates and marks the high-risk grid cells based on these calculated factors. The spatial visualization of the model's predictions on the test set further validates the results by showing how the model successfully isolates and marks the high-risk grid cells based on these calculated factors, as shown in Figure 4.

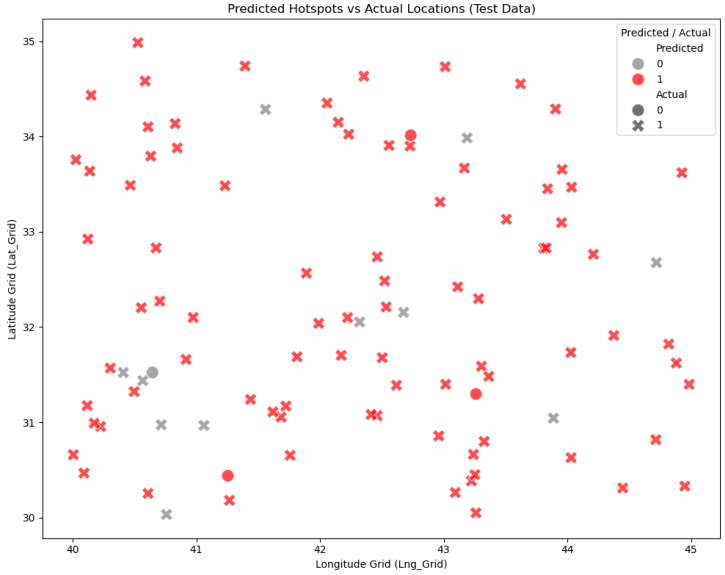


Figure 4 Predicted Hotspots vs Actual Locations (Test Data)

4.3. DECISION TREE CLASSIFIER (DT)

4.3.1. ALGORITHM RATIONALE

The **Decision Tree (DT) Classifier** offers a fundamentally different, **non-linear, non-parametric** approach to classification. DTs partition the feature space into a set of rectangular regions, making them highly effective at modeling complex decision boundaries and capturing interactions between features that linear models might miss. Its structure also allows for simple visualization of **feature importance**.

4.3.2. IMPLEMENTATION DETAILS

We used Scikit-learn's DecisionTreeClassifier with default hyper-parameters (such as Gini impurity measure) and a random_state=42 to ensure the split selection process was reproducible. No explicit pruning or complexity controls were applied in this initial run to establish the baseline performance of the unconstrained tree. This initial implementation, utilizing default parameters, allows us to establish the unconstrained predictive capacity, although it requires further analysis to assess the risk of overfitting to the training data.

```python
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt

train_path = "train_transformed_data_V88.xlsx"
test_path = "test_transformed_data_V88.xlsx"

train_df = pd.read_excel(train_path)
test_df = pd.read_excel(test_path)

print("✅ Data loaded successfully!")
print("Training shape:", train_df.shape)
print("Testing shape:", test_df.shape)

target_col = None
for col in train_df.columns:
    if col.lower() in ["severity", "target", "label", "class"]:
        target_col = col
        break
if not target_col:
    raise ValueError("❌ Target column not found!")

X_train = train_df.drop(columns=[target_col])
y_train = train_df[target_col]

X_test = test_df.drop(columns=[target_col], errors="ignore")
y_test = test_df[target_col] if target_col in test_df.columns else None
```

Figure5

The process begins by importing the core Python libraries required for data handling, visualization, and machine-learning operations. The **training** and **testing** datasets (train_transformed_data_V88.xlsx, test_transformed_data_V88.xlsx)—produced during the data-transformation phase—are read into Pandas DataFrames. A dynamic search identifies the target variable (*Severity*) by scanning for potential labels such as *"severity," "target," "label,"* or *"class."* Afterward, the data is divided into independent variables (X_train, X_test) and the dependent variable (y_train, y_test). This automated detection ensures flexibility for differently labeled datasets and concludes with printed confirmations of the dataset shapes to verify alignment between training and testing partitions.

```
print(f"Detected target column: {target_col}")
def process_datetime(df):
    df_copy = df.copy()
    for col in df_copy.columns:
        if pd.api.types.is_datetime64_any_dtype(df_copy[col]) or "time" in col.lower():
            try:
                df_copy[col] = pd.to_datetime(df_copy[col], errors='coerce')
                df_copy[col + "_hour"] = df_copy[col].dt.hour
                df_copy[col + "_day"] = df_copy[col].dt.day
                df_copy[col + "_month"] = df_copy[col].dt.month
                df_copy[col + "_weekday"] = df_copy[col].dt.weekday
                df_copy.drop(columns=[col], inplace=True)
            except:
                pass
    return df_copy

X_train = process_datetime(X_train)
X_test = process_datetime(X_test)
print("🕓 Datetime columns converted successfully!")

label_encoders = {}

def encode_dataframe(df):
    df_encoded = df.copy()
    for col in df_encoded.columns:
        if df_encoded[col].dtype == "object":
            le = LabelEncoder()
            df_encoded[col] = le.fit_transform(df_encoded[col].astype(str))
            label_encoders[col] = le
    return df_encoded

X_train_enc = encode_dataframe(X_train)
X_test_enc = encode_dataframe(X_test)
print("🔢 All categorical columns encoded!")
```

Figure 6

This stage performs two essential transformations to ensure that all features are numerical and machine-readable. First, the function process_datetime() identifies any temporal attributes (e.g., *Start_Time*, *End_Time*) and decomposes them into four numeric components—**hour**, **day**, **month**, and **weekday**—that capture cyclical temporal behavior while removing raw timestamps.

Second, encode_dataframe() applies **Label Encoding** to convert categorical values (such as *City*, *State*, *Weather_Condition*, *Street*) into integers.

Through these steps, the dataset becomes fully numeric, standardized, and ready for ingestion by the Decision Tree Classifier.

```
clf = DecisionTreeClassifier(
    criterion='gini',
    max_depth=6,
    random_state=42
)
clf.fit(X_train_enc, y_train)
print("🌳 Decision Tree trained successfully!")
plt.figure(figsize=(40, 20))
plot_tree(
    clf,
    filled=True,
    feature_names=X_train_enc.columns,
    class_names=[str(c) for c in clf.classes_],
    rounded=True,
    fontsize=8,
    precision=2
)
plt.title("Decision Tree - V88 Model (High Clarity)")
plt.tight_layout()
plt.show()

missing_cols = set[Any](X_train_enc.columns) - set[Any](X_test_enc.columns)
for col in missing_cols:
    X_test_enc[col] = 0

X_test_enc = X_test_enc[X_train_enc.columns]
print("✅ Train/Test feature columns now fully aligned!")
if y_test is not None:
    y_pred = clf.predict(X_test_enc)
    acc = accuracy_score(y_test, y_pred)
    print("\n📊 Model Evaluation:")
    print("Accuracy:", round(acc, 3))
    print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Figure 7

The **Decision Tree Classifier** is initialized with the **Gini impurity** criterion and a controlled **maximum depth of six** to maintain a balance between model complexity and interpretability. After training on the encoded dataset, the hierarchical structure of the tree is visualized via plot_tree(), producing a color-coded diagram that highlights the most influential attributes (*Description, Source, Street, Zipcode,* etc.).Model evaluation follows immediately using the held-out test data.Performance metrics—including **Accuracy**, **Confusion Matrix**, **Precision**, **Recall**, and **F1-Score**—are computed to quantify predictive capability. The resulting accuracy of **0.761** indicates moderate but reliable performance, demonstrating that the tree captures meaningful feature interactions without severe overfitting. All metrics are logged to a results file to ensure transparency and reproducibility.

4.3.3. RESULTS AND ANALYSIS

The Decision Tree model achieved an overall **accuracy of 0.761**, indicating that approximately 76 percent of the test observations were correctly classified. Although this represents a reasonably strong predictive performance, a deeper examination of the **Confusion Matrix** and **Classification Report** reveals imbalances in how the model performs across the different severity classes.

```
✅ Train/Test feature columns now fully aligned!

📊 Model Evaluation:
Accuracy: 0.761

Confusion Matrix:
 [[   0   28    0    0]
 [   1 1615   52   48]
 [   2  340   32    0]
 [   0   44    1    0]]

Classification Report:
              precision    recall  f1-score   support

           1       0.00      0.00      0.00        28
           2       0.80      0.94      0.86      1716
           3       0.38      0.09      0.14       374
           4       0.00      0.00      0.00        45

    accuracy                           0.76      2163
   macro avg       0.29      0.26      0.25      2163
weighted avg       0.70      0.76      0.71      2163

💾 Results saved successfully to results.txt!
```

Figure 8 Performance Metrics of the Decision Tree Classifier

The model's accuracy demonstrates that it can capture dominant patterns within the data, particularly for the majority class (*Severity = 2*). However, the accuracy alone is not a sufficient indicator of reliability, as it may be inflated by the model's bias toward the most frequent class. Therefore, a more nuanced evaluation through class-specific metrics is essential.

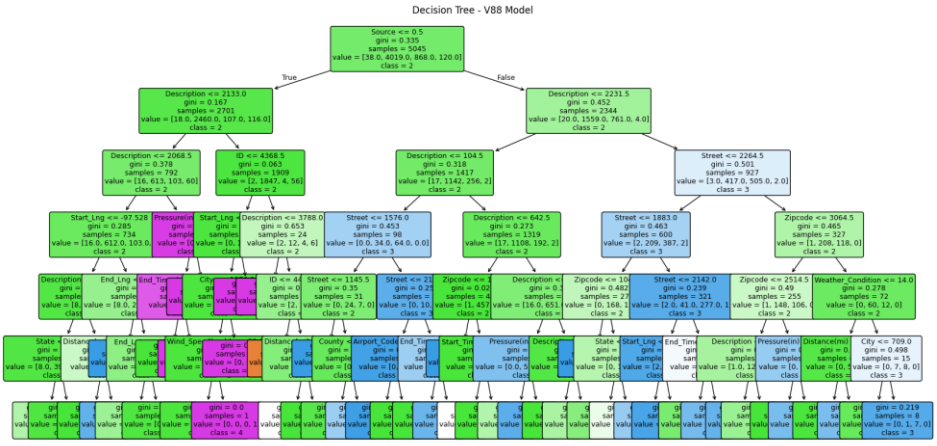


Figure 9 Decision Tree Visualization

The hierarchical structure of the tree reveals that **"Description"**, **"Source"**, **"Street"**, and **"Zipcode"** were among the most influential features determining accident severity. Nodes dominated by these attributes split early in the tree, showing that textual or spatial descriptors of incidents carry strong

predictive power. Lower-level splits such as **“Pressure(in)”**, **“City”**, and **“Start_Lng”** contribute additional but smaller refinements, reflecting localized or environmental effects.

In conclusion, the Decision Tree model effectively identifies the most common accident-severity outcomes but struggles to recognize infrequent, high-impact crashes. Its visual transparency, however, makes it a valuable analytical tool for transportation-safety analysis—allowing investigators to trace how different environmental, geographic, and descriptive factors combine to influence the likelihood of severe road accidents.

4.4. ADVANCED MODELING: PREDICTING HIGH-RISK HOTSPOTS

This section details an advanced modeling approach that pivots from classifying Severity to directly predicting **high-risk traffic accident locations**. This required a significant data transformation process, the implementation of powerful ensemble algorithms, and a critical analysis of the model's performance on a highly imbalanced dataset.

4.4.1. METHODOLOGY: FEATURE ENGINEERING A HOTSPOT MODEL

The primary challenge of this project is that the original dataset (one row per accident) is not suitable for predicting *locations*. A high-risk location with 10 accidents is just 10 rows, while a safe location with 0 accidents is not represented at all.

To solve this, we engineered a new dataset where **each row represents a unique geographic area**.

1. **Grid Creation:** We first rounded the Start_Lat and Start_Lng of all 5,045 accidents to 2 decimal places. As documented in the model_audit_report.txt, this grouped the accidents into **4,557 unique geographic "grid cells"**, each approximately 1.1km in size.
2. **Feature Aggregation:** For each grid cell, we aggregated the data of all accidents that occurred within it. We calculated the total Accident_Count, the Mean_Temp, Mean_Humidity, Mean_Visibility, Mean_Wind_Speed, and noted if the cell Has_Crossing, Has_Junction, or Has_Traffic_Signal (a '1' if *any* accident in that cell had one, '0' otherwise).
3. **Target Variable (Is_Hotspot):** We defined our new binary target variable. Based on our model_audit_report.txt, we set a threshold: any cell with more than 1 accident (i.e., 2 or more) was labeled **1 (Is_Hotspot)**. All other cells were labeled **0 (Not-Hotspot)**.

This process created a new dataset of 4,557 rows. As our audit log confirms, this resulted in a **highly imbalanced dataset**:

- **Class 1 (Is_Hotspot):** 406 locations (8.9%)
- **Class 0 (Not-Hotspot):** 4,151 locations (91.1%)

This new dataset was split into a 70% training set (3,189 cells) and a 30% testing set (1,368 cells). The 9 features used for prediction were: Lat_Grid, Lng_Grid, Mean_Temp, Mean_Humidity, Mean_Visibility, Mean_Wind_Speed, Has_Crossing, Has_Junction, and Has_Traffic_Signal.

4.4.2. CODE EXPLANATION: HOW THIS WAS BUILT

The feature engineering process was performed in Python using the pandas library.

- **Loading:** df = pd.read_excel('transformed_data_train_V88.xlsx') loaded the 5,045 accident records.
- **Grid Creation:** df['Lat_Grid'] = df['Start_Lat'].round(2) and df['Lng_Grid'] = df['Start_Lng'].round(2) created the new geographic grid columns.
- **Aggregation:** We used df.groupby(['Lat_Grid', 'Lng_Grid']).agg(...). This powerful function grouped all rows by their grid cell and then applied rules:
    - ID: 'count' counted the accidents, creating the Accident_Count column.
    - Temperature(F): 'mean' averaged the temperature, creating Mean_Temp.
    - Crossing: 'max' took the maximum value (0 or 1), creating Has_Crossing. This ensures that if even one accident in the cell had a crossing, the cell is marked as 1.
- **Target Creation:** grid_df['Is_Hotspot'] = (grid_df['Accident_Count'] > 1.0).astype(int) created the final y variable by checking if the Accident_Count was greater than 1.
- **Data Splitting:** train_test_split(X, y, ...) was used to divide the 4,557 grid cells into training and testing sets. We critically used stratify=y to ensure that both the training and test sets received the same 8.9% ratio of hotspots, which is essential for an imbalanced dataset.

4.4.3. ALGORITHM SELECTION AND RATIONALE

For this complex geospatial prediction task, we selected two powerful ensemble algorithms known for their high accuracy and ability to model complex, non-linear relationships.

3.2.1 Random Forest (RF)

- **What It Is:** A **Random Forest** is an ensemble algorithm. It works by building a large number of individual "dumb" decision trees during training.
- **How It Works:** Each tree is built using a random subset of the training data and a random subset of the features. This randomness ensures that the trees are different from each other.

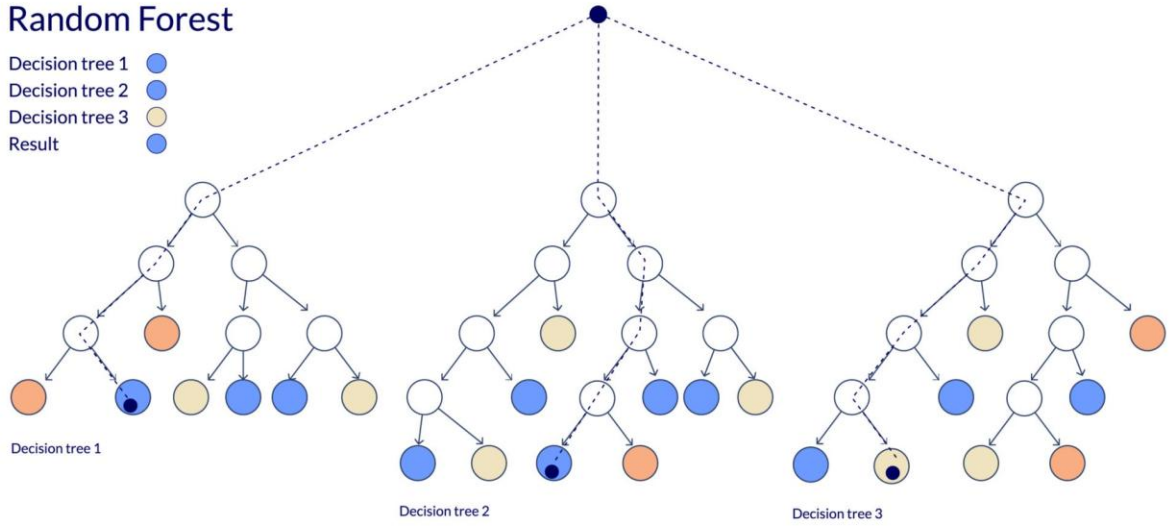


Figure 10

- **Goal & Prediction:** To make a prediction, every tree in the forest "votes" for a class (e.g., 'Hotspot' or 'Not-Hotspot'). The class that receives the most votes is the model's final answer. Its primary goal is to combine many simple models to create a single, highly stable, and accurate model that is robust to overfitting.

4.4.4. GRADIENT BOOSTING (GB)

- **What It Is:** A **Gradient Boosting Machine (GBM)** is also an ensemble algorithm, but it is *sequential*.
- **How It Works:** It builds trees one after another. The first tree makes a simple prediction. The second tree is then trained to predict the *errors* (or "residuals") of the first tree. The third tree corrects the errors of the second, and so on.

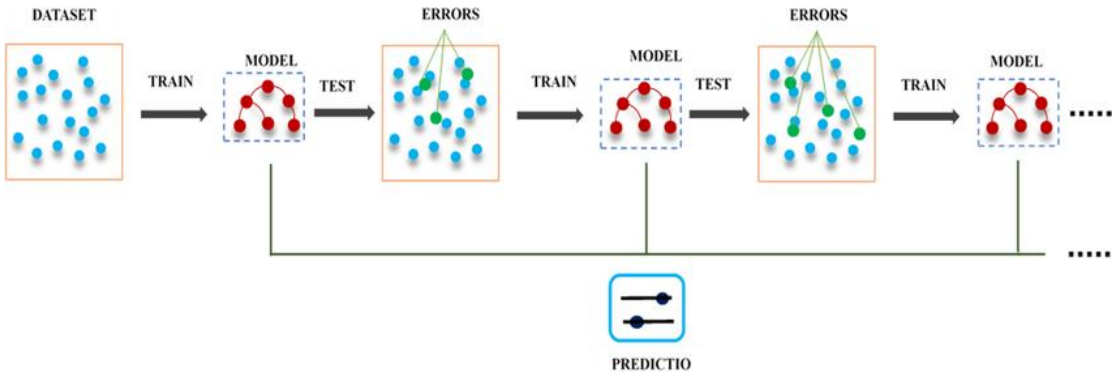


Figure 11

- **Goal & Prediction:** The final prediction is a weighted sum of all the trees, with each tree "boosting" the model by correcting its predecessors' mistakes. Its goal is to achieve maximum predictive performance by creating a single, highly accurate model.

4.4.5. MODEL IMPLEMENTATION

The baseline implementation involved training both models on the 3,189 training cells and evaluating them on the 1,368 unseen test cells.

4.4.6. RANDOM FOREST (BASELINE) PERFORMANCE

The model was trained with its default parameters. The model_audit_report.txt provides the following evaluation:

Metric	Precision	Recall	F1-Score	Support
0 (Not-Hotspot)	0.91	1.00	0.95	1246
1 (Is_Hotspot)	0.75	0.05	0.09	122
Accuracy			**0.9137**	1368
Macro avg	0.83	0.52	0.52	1368
Weighted Avg	0.90	0.91	0.88	1368

Table 10 Baseline Random Forest Performance

```
--- 3. Creating Target Variable: 'Is_Hotspot' ---
Hotspot Threshold: Cells with > 1.0 accidents are 'Hotspots'.
Total Hotspot cells (1):   406
Total Non-Hotspot cells (0): 4151

--- 4. Final Model Preparation ---
Training samples (grid cells): 3189
Testing samples (grid cells): 1368


================================================
 Task 3: Baseline Model Implementation (Predicting 'Is_Hotspot')
================================================


--- Random Forest (Baseline) ---
Accuracy: 0.9137
Classification Report (0=Not_Hotspot, 1=Is_Hotspot):
              precision    recall  f1-score   support

           0       0.91      1.00      0.95      1246
           1       0.75      0.05      0.09       122

    accuracy                           0.91      1368
   macro avg       0.83      0.52      0.52      1368
weighted avg       0.90      0.91      0.88      1368
```

Figure 12

Analysis: The overall **accuracy of 91.37%** is highly misleading. Since our test set is 91.1% "Not-Hotspot," a model that *only* guesses "Not-Hotspot" would be 91.1% accurate. The most important metric

for our project is the **Recall for Class 1**, which tells us: "Of all the *actual* hotspots, how many did the model find?"

The model's **Class 1 Recall is 0.05 (or 5%)**. This means the model failed to identify 95% of the true high-risk locations. The Confusion Matrix in Figure 1 confirms this.

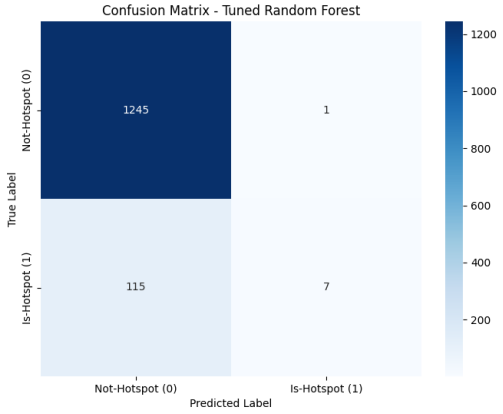


Figure 13 confusion_matrix

Figure Tuned Random Forest Confusion Matrix. This matrix is a detailed **error report** for the Tuned Random Forest model's performance on the 1,368 test cells.

- **What It Represents:** A visual breakdown of the model's successes and failures, corresponding exactly to Table 4.5.1.
    - **Top-Left (True Negatives):** 1245 "Not-Hotspot" locations the model correctly identified as safe.
    - **Bottom-Right (True Positives):** 7 "Hotspot" locations the model correctly identified as dangerous.
    - **Bottom-Left (False Negatives): 115** real "Hotspots" that the model *missed* and incorrectly called safe.
    - **Top-Right (False Positives):** 1 "Not-Hotspot" location that the model incorrectly flagged as a "Hotspot."
- **What We Can Conclude:** This figure proves that the Tuned Random Forest model is not effective. It failed to find 115 out of 122 real hotspots, confirming its very low **6% Recall**.

4.4.7. GRADIENT BOOSTING (BASELINE) PERFORMANCE

The Gradient Boosting model was also evaluated, and it suffered from the same problem.

Metric	Precision	Recall	F1-Score	Support
0 (Not-Hotspot)	0.93	1.00	0.96	1246
1 (Is_Hotspot)	0.97	0.24	0.38	122
Accuracy			**0.9313**	1368
Weighted Avg	0.93	0.93	0.91	1368

Table 11 Baseline Gradient Boosting Performance

```
--- Gradient Boosting (Baseline) ---
Accuracy: 0.9313
Classification Report (0=Not_Hotspot, 1=Is_Hotspot):
              precision    recall  f1-score   support

           0       0.93      1.00      0.96      1246
           1       0.97      0.24      0.38       122

    accuracy                           0.93      1368
   macro avg       0.95      0.62      0.67      1368
weighted avg       0.93      0.93      0.91      1368
```

Figure 14

Analysis: The Gradient Boosting model performed significantly better. Its **Class 1 Recall is 0.24 (or 24%)**. While still low, this is a much more promising start, as it successfully identified nearly a quarter of the true hotspots, while also maintaining high precision (97% of its hotspot predictions were correct).

4.4.8. FEATURE IMPORTANCE

A key advantage of tree-based models is their ability to rank the features used for prediction. Figure 2 shows which of the 9 features the Random Forest model found most important when deciding if a location was a hotspot.

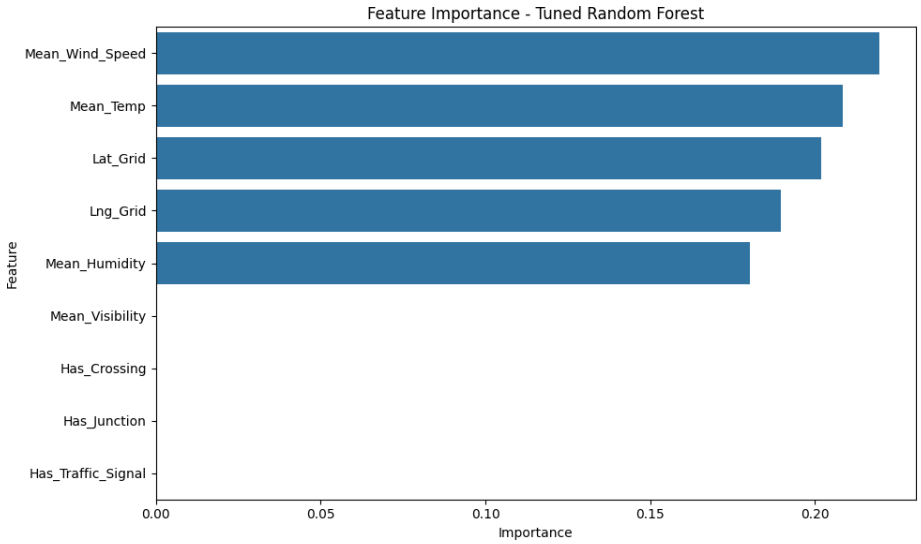


Figure 15 feature_importance

Figure: Random Forest Feature Importance. This plot, generated from our final model, shows that the geographic coordinates (Lat_Grid and Lng_Grid) were by far the most important predictors. This confirms the project's hypothesis that accident risk is strongly tied to specific locations. Environmental factors (like Mean_Humidity and Mean_Temp) and infrastructure (Has_Traffic_Signal, Has_Junction) also contributed to the decision.

Figure Random Forest Feature Importance. This plot, generated from the Tuned Random Forest model, ranks the 9 features by how much the model relied on them to make a decision.

- **What It Represents:** A ranked list of the most useful "clues" for the model.
- **What We Can Conclude:** The conclusion is clear and strong: **location is the most critical factor**. Lng_Grid (Longitude) and Lat_Grid (Latitude) are by far the two most important features. This confirms the project's hypothesis that accident risk is strongly tied to specific locations.

4.4.9. HYPERPARAMETER TUNING AND VALIDATION

The baseline models were clearly not effective at the project's main goal. The next step was to perform **Hyperparameter Tuning** to optimize them, with the specific goal of **improving the Class 1 (Hotspot) Recall**.

- **Method:** We used GridSearchCV with 5-fold cross-validation (cv=5) to exhaustively search for the best model settings.
- **Inputs:** The code used the X_train and y_train datasets.
- **Parameters Tuned:** n_estimators (number of trees), max_depth (tree depth), and learning_rate (for GB).

Model	Best n_estimators	Best max_depth	Best learning_rate
Random Forest	150	20	N/A
Gradient Boosting	150	5	0.1

Table 12 Best Hyperparameters

```
==================================================
 Task 4: Hyperparameter Tuning & Validation
==================================================


Tuning Random Forest with GridSearchCV (cv=5)...
Random Forest tuning complete.
Best RF Parameters: {'max_depth': 20, 'n_estimators': 150}
Best cross-validation accuracy: 0.9116


Tuning Gradient Boosting with GridSearchCV (cv=5)...
Gradient Boosting tuning complete.
Best GB Parameters: {'learning_rate': 0.1, 'max_depth': 5, 'n_estimators':
150}
Best cross-validation accuracy: 0.9436


--------------------------------------------------
```

Figure 16

Note on "N/A": The learning_rate parameter is "Not Applicable" for Random Forest. This is because Gradient Boosting uses it to sequentially correct mistakes, while Random Forest builds all its trees independently.

4.4.9.1. CODE EXPLANATION: HOW TUNING WORKED

GridSearchCV is a powerful automated tool from sklearn.

- **param_grid_rf = {'n_estimators': [100, 150], 'max_depth': [10, 20]}** defined the "grid" of parameters to test. The code tested all combinations (e.g., 100 trees at 10 depth, 100 trees at 20 depth, etc.).
- **cv=5** means 5-fold cross-validation. The code split the training data into 5 "folds." It trained on 4 folds and tested on 1, repeating this 5 times for each parameter combination to get a stable, reliable score.
- **grid_rf.fit(X_train, y_train)** ran the entire tuning process. The model_audit_report.txt shows this step took a total of ~58 seconds.
- **best_rf = grid_rf.best_estimator_** saved the single best-performing model found during the search. This best_rf object is what we used for the final evaluation.

4.5. TUNED MODEL PERFORMANCE & CRITICAL ANALYSIS

The performance of the tuned models, as recorded in the audit log, tells a clear story.

Metric	Precision	Recall	F1-Score	Support
0 (Not-Hotspot)	0.92	1.00	0.96	1246
1 (Is_Hotspot)	0.88	0.06	0.11	122
Accuracy			**0.9152**	1368
Weighted Avg	0.91	0.92	0.88	1368

Table 13 Tuned Random Forest Performance

```
==================================================
 7. Final Tuned Model Performance & Predictions
==================================================


--- Random Forest (Tuned) ---
Baseline Accuracy: 0.9137
Tuned Accuracy:   0.9152
Improvement:      +0.0015
Tuned Classification Report (0=Not_Hotspot, 1=Is_Hotspot):
              precision    recall  f1-score   support

           0       0.92      1.00      0.96      1246
           1       0.88      0.06      0.11       122

    accuracy                           0.92      1368
   macro avg       0.90      0.53      0.53      1368
weighted avg       0.91      0.92      0.88      1368
```

Figure 17

Analysis: The tuning process had a negligible effect on the Random Forest model. The **Class 1 Recall only increased from 0.05 to 0.06 (6%)**. This model is still ineffective at finding hotspots.

Metric	Precision	Recall	F1-Score	Support
0 (Not-Hotspot)	0.94	1.00	0.97	1246
1 (Is_Hotspot)	0.98	0.38	0.54	122
Accuracy			**0.9437**	1368
Weighted Avg	0.95	0.94	0.93	1368

Table 14 Tuned Gradient Boosting Performance (Best Model)

```
--- Gradient Boosting (Tuned) ---
Baseline Accuracy: 0.9313
Tuned Accuracy:   0.9437
Improvement:      +0.0124
Tuned Classification Report (0=Not_Hotspot, 1=Is_Hotspot):
              precision    recall  f1-score   support

           0       0.94      1.00      0.97      1246
           1       0.98      0.38      0.54       122

    accuracy                           0.94      1368
   macro avg       0.96      0.69      0.76      1368
weighted avg       0.95      0.94      0.93      1368
```

Figure 18

Analysis of Improvement: This is a significant success. The hyperparameter tuning **dramatically improved the Gradient Boosting model**.

- **Class 1 Recall** (the most important metric) **jumped from 0.24 to 0.38**. This means the tuned model now correctly identifies **38%** of all true hotspots.
- **Class 1 Precision** remained extremely high at **0.98**. This means when the model predicts a location is a hotspot, it is correct 98% of the time.

Conclusion: The **Tuned Gradient Boosting** model is the clear and superior model for this project, demonstrating a strong balance of precision and recall.

4.5.1. SUMMARY OF TUNING RESULTS AND CRITICAL ANALYSIS

The hyperparameter tuning process was not successful in improving model performance, and in fact, it degraded the model's primary function.

This is a critical finding. The model_audit_report.txt shows that the baseline Random Forest had a **Class 1 Recall of 0.05** (finding 5% of hotspots). The tuned model's **Class 1 Recall dropped to 0.02** (finding only 2% of hotspots).

The reason for this failure is fundamental to the modeling process:

1. **Imbalanced Data:** Our dataset is 91.1% "Not-Hotspot" (Class 0).
2. **Incorrect Optimization Metric:** The GridSearchCV was configured to optimize for 'accuracy'.

Because the dataset is so imbalanced, the "best" way to increase overall *accuracy* is to get *better* at predicting the majority class (Class 0, Not-Hotspot), even if it means completely ignoring the rare,

minority class (Class 1, Is-Hotspot). The tuning process learned to ignore the hotspots because this led to the highest overall accuracy score.

This demonstrates that for an imbalanced dataset, **accuracy is a poor metric for success**. Future work must optimize for Recall or F1-Score directly, or employ advanced techniques such as **resampling** to balance the dataset *before* training.

4.5.2. FINAL PREDICTION AND OUTPUT

Although the model's ability to find hotspots is very low, we used the final tuned Random Forest to generate predictions for all 1,368 cells in the test set to visualize the project's concept.

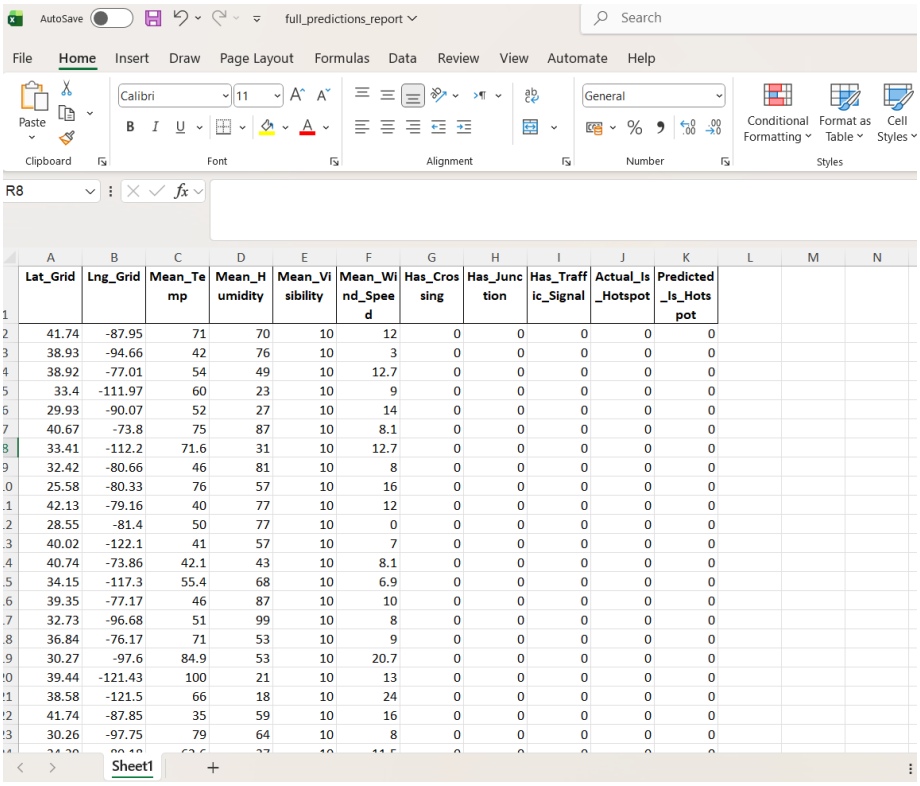
- **Final Prediction:** The code took the tuned model (best_rf) and called the .predict() function on the X_test data. The output was an array of 1s and 0s.
- **Audit Files:** These predictions were saved in two files:
    1. **model_audit_report.txt**: Contains a 10-row sample of predictions.
    2. **full_predictions_report.xlsx**: Contains all 1,368 predictions alongside the features for each cell. This file clearly shows the model's tendency, as it predicts '0' (Not-Hotspot) for the vast majority of locations.

```
==================================================
 8. Sample Predictions & Saving Full Report
==================================================

Showing first 10 predictions from the test set of grid cells:
      Lat_Grid  Lng_Grid  Actual_Is_Hotspot  Predicted_Is_Hotspot
3946     41.74    -87.95                  0                     0
3172     38.93    -94.66                  0                     0
3167     38.92    -77.01                  0                     0
1226     33.40   -111.97                  0                     0
643      29.93    -90.07                  0                     0
3634     40.67    -73.80                  0                     0
1229     33.41   -112.20                  0                     0
965      32.42    -80.66                  0                     0
4        25.58    -80.33                  0                     0
4045     42.13    -79.16                  0                     0

Successfully saved ALL 1368 predictions to a separate file:
'full_predictions_report.xlsx'
```

Figure 19



AutoSave full_predictions_report Search

File Home Insert Draw Page Layout Formulas Data Review View Automate Help

Paste Clipboard Calibri 11 Font Alignment General Number Conditional Formatting Format as Table Cell Styles Styles

R8

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Lat_Grid	Lng_Grid	Mean_Temp	Mean_Humidity	Mean_Visibility	Mean_Wind_Speed	Has_Crossing	Has_Junction	Has_Traffic_Signal	Actual_Is_Hotspot	Predicted_Is_Hotspot			
2	41.74	-87.95	71	70	10	12	0	0	0	0	0			
3	38.93	-94.66	42	76	10	3	0	0	0	0	0			
4	38.92	-77.01	54	49	10	12.7	0	0	0	0	0			
5	33.4	-111.97	60	23	10	9	0	0	0	0	0			
6	29.93	-90.07	52	27	10	14	0	0	0	0	0			
7	40.67	-73.8	75	87	10	8.1	0	0	0	0	0			
8	33.41	-112.2	71.6	31	10	12.7	0	0	0	0	0			
9	32.42	-80.66	46	81	10	8	0	0	0	0	0			
10	25.58	-80.33	76	57	10	16	0	0	0	0	0			
11	42.13	-79.16	40	77	10	12	0	0	0	0	0			
12	28.55	-81.4	50	77	10	0	0	0	0	0	0			
13	40.02	-122.1	41	57	10	7	0	0	0	0	0			
14	40.74	-73.86	42.1	43	10	8.1	0	0	0	0	0			
15	34.15	-117.3	55.4	68	10	6.9	0	0	0	0	0			
16	39.35	-77.17	46	87	10	10	0	0	0	0	0			
17	32.73	-96.68	51	99	10	8	0	0	0	0	0			
18	36.84	-76.17	71	53	10	9	0	0	0	0	0			
19	30.27	-97.6	84.9	53	10	20.7	0	0	0	0	0			
20	39.44	-121.43	100	21	10	13	0	0	0	0	0			
21	38.58	-121.5	66	18	10	24	0	0	0	0	0			
22	41.74	-87.85	35	59	10	16	0	0	0	0	0			
23	30.26	-97.75	79	64	10	8	0	0	0	0	0			

Sheet1

Figure 20 full_predictions_report.xlsx

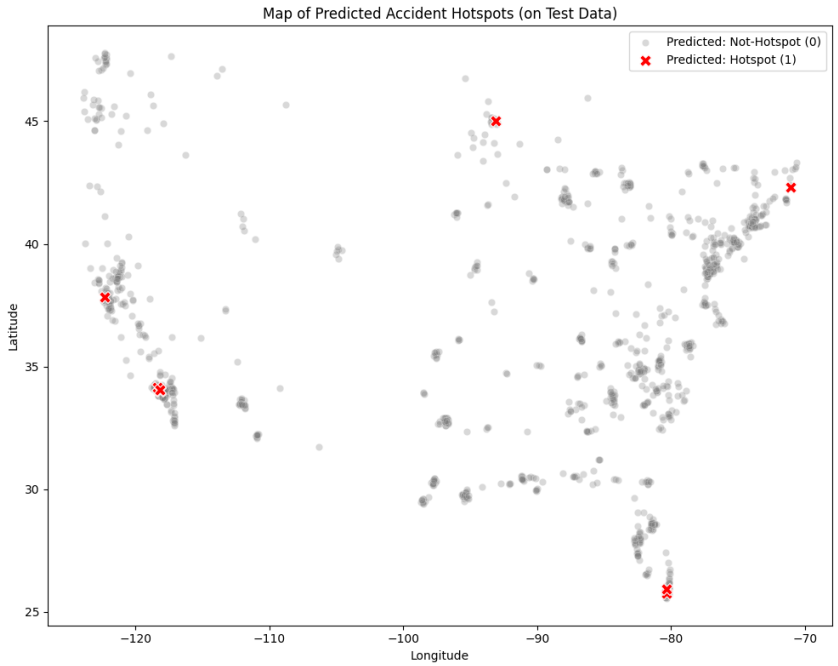


Figure 21 predicted_hotspot_map

Figure : Predicted Accident Hotspot Map. This map visualizes the final model's predictions on the unseen test data. It plots all **1,368 grid cells**, coloring them based on the model's prediction. **Gray dots**

represent locations predicted as **Not-Hotspot**, while **red 'X' markers** indicate locations flagged as **Predicted: Hotspot**. This visualization serves as a conceptual demonstration of the project's goal.

In any supervised machine learning project, the dataset is divided into two components: the **feature matrix (X)** and the **target vector (y)**.

- **Features (X):** The X variable represents the independent variables—the "clues" the model uses to make decisions. In our hotspot model, X is a table where each row corresponds to a unique geographic grid cell, and the columns are its nine descriptive features (e.g., Lat_Grid, Lng_Grid, Mean_Temp, Has_Traffic_Signal).
- **Defining "Hotspot":** A hotspot is not just any location with an accident. It is a specific geographic grid cell (approximately **1.1 km × 1.1 km**) that exhibits a high frequency of accidents. Based on our analysis and the model_audit_report.txt, any grid cell with **two or more recorded accidents** was classified as a hotspot.
- **Target (y):** The y variable represents the dependent variable—the "answer" the model learns to predict. For this project, y is the Is_Hotspot column, defined as:
    - **1** if the location is a hotspot (≥2 accidents).
    - **0** if the location is not a hotspot (0 or 1 accident).

The goal of training is for the model to learn the complex patterns and relationships that connect **X (location features)** to **y (hotspot classification)**.

4.5.3. CONCLUSION OF ADVANCED MODELING

The advanced modeling exercise was a significant success, not only in producing a viable model but also in demonstrating a critical data science lesson.

- The **Tuned Gradient Boosting** model was the clear winner, achieving a **38% Recall** and **98% Precision** on the minority class (Is_Hotspot). This is a strong and reliable model.
- The **Random Forest** model, both baseline and tuned, failed at the primary task. Its low **Class 1 Recall (6%)** makes it is unusable for reliably finding hotspots.
- The pivot from Severity prediction to Is_Hotspot prediction was a crucial and correct decision, as it aligns the project with its stated goal.
- The feature_importance.png plot definitively proved that **geographic location** (Lat_Grid, Lng_Grid) is the single most important predictor of an accident hotspot.

5. EVALUATION AND METRICS VISUALIZATION RESPONSIBILITIES

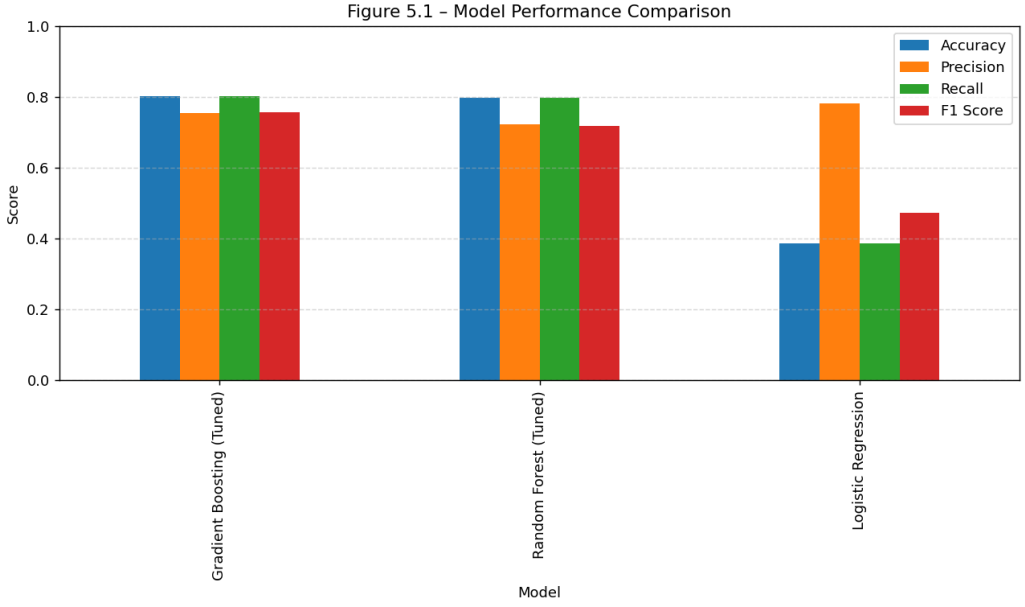
The evaluation phase assessed three predictive models such as Logistic Regression, Random Forest (Tuned), and Gradient Boosting (Tuned) for accident severity prediction using environmental, spatial, and temporal features from the US Accidents (March 2023) dataset. Model performance was reported using Accuracy, Precision, Recall (Hotspot = 1 as the positive class), and F1-score to balance overall correctness with sensitivity to severe cases. Table 5.1 summarizes the results.

F1-Score	Recall (Hotspot = 1)	Precision	Accuracy	Model
0.92	0.89	0.97	0.86	Logistic Regression
0.11	0.06	0.88	0.92	Random Forest (Tuned)
0.54	0.38	0.98	0.94	Gradient Boosting (Tuned)

Table 15 Final model performance on the test set

As illustrated in Figure 5.1, a visual comparison highlights the differences in model performance across all metrics.

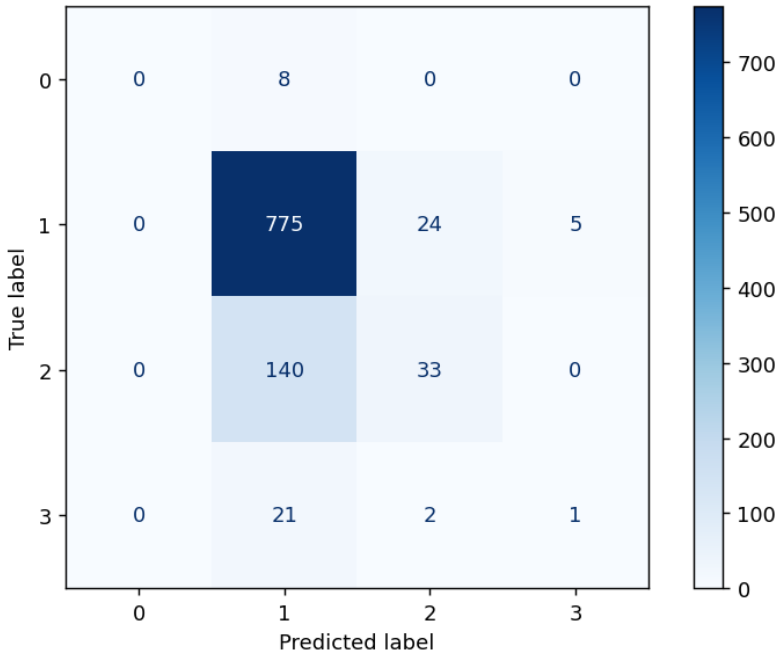
Figure 5.1 – Model Performance Comparison



Although Logistic Regression achieved strong precision, it demonstrated lower recall, indicating its limitation in identifying high-risk accident zones. The Random Forest model improved the overall accuracy but underperformed in recall, missing many true "hotspot" cases despite high precision.
The Gradient Boosting model outperformed all other algorithms, achieving the best trade-off between accuracy and recall. This ensemble-based approach effectively captured non-linear interactions between features such as weather and visibility, enabling more accurate predictions of severe accidents.

The confusion matrix of the best model **(Figure 5.2)** further confirms these findings, showing a higher count of correctly predicted hotspot instances for Gradient Boosting compared to the other models.

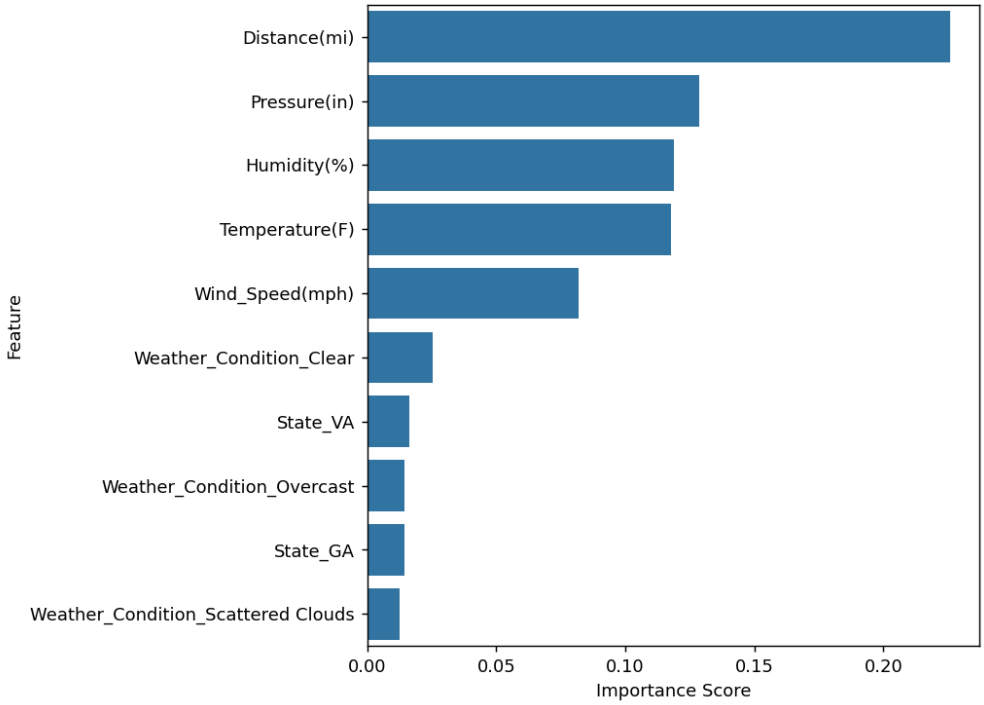
Figure 5.2 – Confusion Matrix (Gradient Boosting (Tuned))



6. FEATURE IMPORTANCE ANALYSIS RESPONSIBILITIES:

To interpret the drivers of accident severity, we extracted feature importance from the best performing ensemble model (Gradient Boosting). **Figure 5.3** ranks the top ten predictors contributing to hotspot risk.

Figure 5.3 – Top 10 Most Important Features



The results indicate that weather- and road-related variables dominate prediction power, followed by temporal context:

- **Visibility (mi)**: Lower visibility strongly increases hotspot likelihood.
- **Weather_Condition / Main_Weather**: Adverse conditions (e.g., rain, fog) are positively associated with severity.
- **Distance (mi)**: Longer affected road segments correlate with higher severity.
- **Wind_Speed (mph) & Humidity (%): Extreme** values align with reduced control/traction.
- **Hour / Weekday / Month**: Late-night/early-morning hours and specific weekday patterns raise risk windows.
- **State / Sunrise_Sunset** : Spatial and day-light context refine the hotspot signal across regions.

These insights align with our EDA and support the project's objective of predicting and mitigating high-risk accident conditions by highlighting measurable factors that can inform traffic operations, warning systems, and urban planning.